

Explication complète du projet PBxcom

Guide de compréhension de A à Z : architecture, parcours utilisateur, fichiers frontend, fichiers backend, et réponses prêtes pour l'encadrant.

1. Résumé du projet

- **Nom du projet** : PBxcom support platform.
- **Objectif** : permettre aux clients de créer des tickets de support, et aux administrateurs/employés PBxcom de les consulter, les traiter, les commenter, les imprimer et suivre leur état.
- **Frontend** : React avec Vite, Tailwind CSS, Zustand, React Router, Axios, Framer Motion et Lucide Icons.
- **Backend** : Node.js avec Express, MongoDB/Mongoose, JWT, bcrypt, Nodemailer et middleware de sécurité.
- **État actuel** : le frontend est utilisable même sans serveur grâce à un stockage local dans le navigateur. Le backend est prêt pour une vraie base MongoDB et de vrais emails.

2. Architecture générale

Couche	Rôle simple	Exemples de fichiers
Interface	Pages, formulaires, tableaux, boutons et navigation.	src/pages, src/components
État local	Stocke utilisateur, tickets, thème, langue et notifications.	src/store
Services	Appels API, lecture des fichiers joints, villes marocaines.	src/services
API backend	Routes Express, sécurité, MongoDB, emails.	backend/src
Configuration	Vite, ESLint, Vercel, variables d'environnement.	package.json, vite.config.js, vercel.json

Flux principal

- Un client crée un compte ou se connecte.
- Il remplit un formulaire de ticket avec ses informations, la ville, la description du problème et éventuellement des pièces jointes.
- Le ticket est enregistré localement si l'API n'est pas disponible, ou envoyé au backend si l'API répond.
- Un admin ou employé voit le ticket dans son tableau de bord, le marque comme vu, change le statut, ajoute une note interne ou répond au client.
- Les notifications sont simulées dans le frontend, et le backend contient aussi l'envoi réel par email via SMTP.

3. Technologies utilisées

- **React** : construit l'interface sous forme de composants réutilisables.
- **Vite** : lance le projet rapidement en développement et construit le dossier final **dist**.
- **Tailwind CSS** : gère le design directement avec des classes utilitaires.
- **Zustand** : sert de magasin global pour partager les données entre pages.
- **React Router** : gère les URLs comme /login, /client, /admin/tickets.
- **Axios** : prépare les appels HTTP vers le backend.

- **Express** : expose les endpoints API côté serveur.
- **MongoDB/Mongoose** : structure les données utilisateurs et tickets.
- **JWT + bcrypt** : sécurisent la connexion : token d'accès et mot de passe hashé.
- **Nodemailer** : permet d'envoyer des emails de notification ou de réinitialisation.

4. Explication fichier par fichier : racine du projet

README.md

Rôle : Fichier d'information initial de Vite.

Ce qu'on a fait : Il explique le template React/Vite de base. Il n'a pas encore été personnalisé pour PBxcom.

À dire à l'encadrant : Dire que c'est le README généré au départ, mais que l'application réelle se trouve dans src et backend.

package.json

Rôle : Déclare le projet frontend et ses dépendances.

Ce qu'on a fait : On y a les scripts dev, build, lint et preview, plus React, Vite, Tailwind, Zustand, Axios, etc.

À dire à l'encadrant : Dire que c'est la carte d'identité du frontend.

package-lock.json

Rôle : Verrouille les versions exactes installées.

Ce qu'on a fait : Il est généré automatiquement par npm pour garantir les mêmes versions sur une autre machine.

À dire à l'encadrant : Dire qu'on ne le modifie pas manuellement.

index.html

Rôle : Point d'entrée HTML.

Ce qu'on a fait : Il contient la div root et charge src/main.jsx.

À dire à l'encadrant : Dire que React injecte toute l'application dans #root.

vite.config.js

Rôle : Configuration de Vite.

Ce qu'on a fait : Active le plugin React et le plugin Tailwind.

À dire à l'encadrant : Dire que Vite sait ainsi compiler JSX et CSS Tailwind.

eslint.config.js

Rôle : Configuration de contrôle qualité du code.

Ce qu'on a fait : Définit les règles pour les fichiers React et Node backend.

À dire à l'encadrant : Dire qu'ESLint aide à détecter les erreurs avant l'exécution.

vercel.json

Rôle : Configuration de déploiement Vercel.

Ce qu'on a fait : Indique comment construire l'application et redirige les routes vers l'application React.

À dire à l'encadrant : Dire que c'est utile pour que /client ou /admin marche même après rafraîchissement.

.env

Rôle : Variables locales du frontend.

Ce qu'on a fait : Contient les valeurs privées locales. Le contenu ne doit pas être publié.

À dire à l'encadrant : Dire que les secrets restent dans .env, pas dans le code.

.gitignore

Rôle : Liste les fichiers à ne pas versionner.

Ce qu'on a fait : Ignore normalement node_modules, dist, fichiers système, variables privées, etc.

À dire à l'encadrant : Dire qu'il protège le dépôt contre les fichiers inutiles ou sensibles.

dist/

Rôle : Version compilée du frontend.

Ce qu'on a fait : Dossier généré après npm run build.

À dire à l'encadrant : Dire que c'est ce qu'on peut déployer, mais qu'on ne travaille pas directement dedans.

node_modules/

Rôle : Bibliothèques installées.

Ce qu'on a fait : Dossier généré par npm install.

À dire à l'encadrant : Dire qu'il est lourd, généré automatiquement, et non écrit à la main.

5. Frontend : démarrage et structure globale

[src/main.jsx](#)

Rôle : Démarre l'application React.

Ce qu'on a fait : Il importe le CSS global, importe App, puis rend App dans l'élément HTML #root avec StrictMode.

À dire à l'encadrant : Dire que c'est le premier fichier JavaScript exécuté côté frontend.

[src/app/App.jsx](#)

Rôle : Définit toutes les routes de l'application.

Ce qu'on a fait : Il branche les pages auth, client, admin, la protection par rôle, le layout et les toasts.

À dire à l'encadrant : Dire que c'est le plan de navigation du projet.

[src/index.css](#)

Rôle : Style global de l'application.

Ce qu'on a fait : Importe Tailwind, définit la variante dark, les couleurs PBxcom, l'arrière-plan animé et les règles d'impression.

À dire à l'encadrant : Dire que ce fichier donne l'identité visuelle et permet d'imprimer proprement les tickets.

6. Frontend : services et données

[src/services/api.js](#)

Rôle : Centralise la connexion au backend.

Ce qu'on a fait : Crée une instance Axios, ajoute automatiquement le token JWT dans les headers, traduit les erreurs et convertit un fichier en pièce jointe base64.

À dire à l'encadrant : Dire que tout appel serveur passe par ce service pour éviter de répéter le code.

[src/services/moroccanCities.js](#)

Rôle : Liste les villes marocaines.

Ce qu'on a fait : Fournit un tableau de villes et une fonction getMoroccanCities utilisée dans le formulaire de ticket.

À dire à l'encadrant : Dire que cela évite la saisie libre de la ville et rend le formulaire plus propre.

[src/data/seedData.js](#)

Rôle : Données de départ.

Ce qu'on a fait : Déclare les statuts possibles, le compte admin par défaut et une liste de tickets vide.

À dire à l'encadrant : Dire que le compte admin local permet de tester sans backend.

[src/utils/formatters.js](#)

Rôle : Fonctions d'affichage.

Ce qu'on a fait : Formate les dates, calcule les initiales d'un nom et choisit la couleur d'un badge selon le statut.

À dire à l'encadrant : Dire que ces fonctions évitent de répéter la logique d'affichage partout.

[src/utils/tickets.js](#)

Rôle : Fonctions utilitaires des tickets.

Ce qu'on a fait : Formate les numéros de tickets, extrait leur valeur numérique, calcule le prochain numéro et trie les tickets.

À dire à l'encadrant : Dire que cela garantit des tickets ordonnés comme 00001, 00002, etc.

7. Frontend : stores Zustand

[src/store/authStore.js](#)

Rôle : Gère l'authentification côté frontend.

Ce qu'on a fait : Contient login, register, updateProfile, forgotPassword, resetDemoData et logout. Il tente d'abord l'API réelle, puis bascule sur le mode local si le serveur ne répond pas.

À dire à l'encadrant : Dire que le projet fonctionne en démonstration sans base de données grâce à ce fallback local.

[src/store/ticketStore.js](#)

Rôle : Gère les tickets.

Ce qu'on a fait : Charge, crée, modifie le statut, marque comme vu, ajoute notes, messages, pièces jointes et retrouve un ticket par ID. Comme authStore, il utilise l'API si possible puis le stockage local.

À dire à l'encadrant : Dire que c'est le coeur métier du frontend.

[src/store/notificationStore.js](#)

Rôle : Simule les notifications email dans l'interface.

Ce qu'on a fait : Crée des notifications avec destinataire, sujet, message, ticketId et état lu/non lu.

À dire à l'encadrant : Dire que c'est une simulation visible côté frontend, pendant que le backend contient l'email réel.

[src/store/toastStore.js](#)

Rôle : Gère les messages courts de succès ou erreur.

Ce qu'on a fait : Ajoute et retire les toasts affichés en bas de l'écran.

À dire à l'encadrant : Dire que cela améliore l'expérience utilisateur après une action.

[src/store/themeStore.js](#)

Rôle : Gère le thème clair, sombre ou système.

Ce qu'on a fait : Stocke le choix utilisateur et applique la classe dark sur la balise html.

À dire à l'encadrant : Dire que le thème reste mémorisé dans le navigateur.

[src/store/languageStore.js](#)

Rôle : Gère les langues.

Ce qu'on a fait : Contient les textes anglais, français et arabes, le choix de langue, et change aussi dir=rtl pour l'arabe.

À dire à l'encadrant : Dire que l'application est préparée pour une interface multilingue.

8. Frontend : composants layout

[src/components/layout/AuthLayout.jsx](#)

Rôle : Layout des pages de connexion.

Ce qu'on a fait : Affiche la partie visuelle PBxcom, le logo, le texte marketing, le switch thème et l'emplacement des formulaires auth via Outlet.

À dire à l'encadrant : Dire que ce composant évite de répéter la même structure sur login/register/forgot/reset.

[src/components/layout/DashboardLayout.jsx](#)

Rôle : Layout principal après connexion.

Ce qu'on a fait : Crée la sidebar, le header, la recherche admin, le centre de notifications, l'affichage utilisateur, logout, responsive mobile et l'Outlet des pages.

À dire à l'encadrant : Dire que c'est le cadre commun de toutes les pages client et admin.

[src/components/layout/ProtectedRoute.jsx](#)

Rôle : Protection des routes.

Ce qu'on a fait : Redirige vers /login si aucun utilisateur n'est connecté, et bloque l'accès si le rôle n'est pas autorisé.

À dire à l'encadrant : Dire que c'est la première sécurité côté interface, complétée par le backend.

[src/components/layout/ThemeSwitcher.jsx](#)

Rôle : Sélecteur de thème.

Ce qu'on a fait : Affiche trois boutons avec icônes : clair, sombre, système.

À dire à l'encadrant : Dire qu'il modifie le store de thème, pas seulement un style local.

9. Frontend : composants UI

[src/components/ui/Button.jsx](#)

Rôle : Bouton réutilisable.

Ce qu'on a fait : Propose des variantes primary, secondary, ghost et danger avec le même style de base.

À dire à l'encadrant : Dire qu'un seul composant garantit des boutons cohérents.

[src/components/ui/Input.jsx](#)

Rôle : Champs de formulaire.

Ce qu'on a fait : Contient Input, PasswordInput avec affichage/masquage, Textarea et gestion des erreurs.

À dire à l'encadrant : Dire que les formulaires deviennent plus simples et uniformes.

[src/components/ui/Select.jsx](#)

Rôle : Liste déroulante réutilisable.

Ce qu'on a fait : Gère label, required, erreur et style Tailwind.

À dire à l'encadrant : Dire qu'elle est utilisée pour statut, rôle, langue et ville.

[src/components/ui/Card.jsx](#)

Rôle : Bloc visuel réutilisable.

Ce qu'on a fait : Encadre les zones importantes avec bordure, fond, ombre et hover.

À dire à l'encadrant : Dire que c'est la base des panneaux de dashboard.

[src/components/ui/Badge.jsx](#)

Rôle : Petit indicateur coloré.

Ce qu'on a fait : Affiche un statut avec la couleur calculée par statusTone.

À dire à l'encadrant : Dire qu'il rend l'état d'un ticket lisible rapidement.

[src/components/ui/PageHeader.jsx](#)

Rôle : En-tête de page.

Ce qu'on a fait : Standardise eyebrow, titre, description et bouton d'action.

À dire à l'encadrant : Dire qu'il donne une cohérence aux pages.

[src/components/ui/EmptyState.jsx](#)

Rôle : État vide.

Ce qu'on a fait : Affiche une icône, un titre, une description et éventuellement une action.

À dire à l'encadrant : Dire qu'on évite les pages vides incompréhensibles.

[src/components/ui/ToastHost.jsx](#)

Rôle : Affichage des notifications courtes.

Ce qu'on a fait : Écoute le toastStore et affiche les messages animés avec succès ou erreur.

À dire à l'encadrant : Dire que l'utilisateur reçoit un retour immédiat.

[src/components/ui/Skeleton.jsx](#)

Rôle : Placeholder de chargement.

Ce qu'on a fait : Définit un bloc animé pour représenter un chargement.

À dire à l'encadrant : Dire qu'il est prêt si on ajoute des chargements visuels.

[src/components/ui/BrandLogo.jsx](#)

Rôle : Logo PBxcom.

Ce qu'on a fait : Affiche un logo différent selon le thème clair/sombre et supporte un mode compact.

À dire à l'encadrant : Dire que le branding reste lisible dans tous les thèmes.

10. Frontend : composants tickets

[src/components/tickets/TicketTable.jsx](#)

Rôle : Tableau de tickets.

Ce qu'on a fait : Trie les tickets, affiche ticket, client, problème, statut, priorité, date et lien d'ouverture.

À dire à l'encadrant : Dire que c'est la vue principale pour consulter une liste de tickets.

[src/components/tickets/TicketCard.jsx](#)

Rôle : Carte ticket.

Ce qu'on a fait : Affiche une version compacte d'un ticket avec badge, description, société et date.

À dire à l'encadrant : Dire qu'il peut servir pour une vue en cartes.

[src/components/tickets/TicketTimeline.jsx](#)

Rôle : Historique du ticket.

Ce qu'on a fait : Affiche les événements du ticket dans une timeline verticale.

À dire à l'encadrant : Dire qu'il permet de justifier la traçabilité des actions.

[src/components/tickets/PrintableTicket.jsx](#)

Rôle : Version imprimable.

Ce qu'on a fait : Présente le ticket avec informations client, statut, priorité, sujet et description dans une zone compatible impression.

À dire à l'encadrant : Dire que l'admin peut imprimer une fiche de service claire.

11. Frontend : pages d'authentification

[src/pages/auth/Login.jsx](#)

Rôle : Page connexion.

Ce qu'on a fait : Formulaire email/mot de passe, appelle login, affiche toast et redirige client vers /client ou admin/employé vers /admin.

À dire à l'encadrant : Dire que la redirection dépend du rôle.

[src/pages/auth/Register.jsx](#)

Rôle : Page création compte client.

Ce qu'on a fait : Crée un compte client avec nom, société, email et mot de passe puis redirige vers l'espace client.

À dire à l'encadrant : Dire que seuls les clients s'inscrivent eux-mêmes.

[src/pages/auth/ForgotPassword.jsx](#)

Rôle : Page mot de passe oublié.

Ce qu'on a fait : Demande l'email et appelle forgotPassword pour envoyer les instructions si le compte existe.

À dire à l'encadrant : Dire que la réponse reste volontairement générique pour éviter de révéler les emails existants.

[src/pages/auth/ResetPassword.jsx](#)

Rôle : Page nouveau mot de passe.

Ce qu'on a fait : Lit le token dans l'URL et appelle l'API /auth/reset-password avec le nouveau mot de passe.

À dire à l'encadrant : Dire que cette page dépend du lien envoyé par email.

[src/pages/NotFound.jsx](#)

Rôle : Page 404.

Ce qu'on a fait : Affiche une page simple si l'URL n'existe pas ou si un ticket est inaccessible.

À dire à l'encadrant : Dire qu'elle protège aussi contre l'accès à un ticket non autorisé côté client.

12. Frontend : pages client

[src/pages/client/ClientDashboard.jsx](#)

Rôle : Dashboard client.

Ce qu'on a fait : Réutilise directement ClientTickets pour afficher l'historique comme page d'accueil client.

À dire à l'encadrant : Dire que l'espace client est centré sur l'historique des demandes.

[src/pages/client/ClientTickets.jsx](#)

Rôle : Historique client.

Ce qu'on a fait : Filtre les tickets de l'utilisateur connecté, permet recherche et filtre par statut, puis affiche TicketTable ou EmptyState.

À dire à l'encadrant : Dire qu'un client ne voit que ses propres tickets.

[src/pages/client/CreateTicket.jsx](#)

Rôle : Création de ticket.

Ce qu'on a fait : Formulaire complet : identité client, société, marché, facture, téléphone, email, ville, description, pièces jointes. Valide les champs, crée le ticket et notifie les admins.

À dire à l'encadrant : Dire que c'est le point principal d'entrée des demandes support.

[src/pages/client/TicketDetail.jsx](#)

Rôle : Détail ticket côté client.

Ce qu'on a fait : Affiche description, informations client, historique, conversation, réponse et pièces jointes. Refuse l'accès si le ticket n'appartient pas au client.

À dire à l'encadrant : Dire que le client peut suivre et enrichir sa demande.

[src/pages/client/ClientSettings.jsx](#)

Rôle : Paramètres client.

Ce qu'on a fait : Permet de modifier profil, email, mot de passe, langue, et de réinitialiser les données de démonstration locales.

À dire à l'encadrant : Dire que cette page montre la personnalisation du compte.

13. Frontend : pages admin

[src/pages/admin/AdminDashboard.jsx](#)

Rôle : Tableau de bord admin.

Ce qu'on a fait : Calcule les statistiques principales, affiche les activités récentes et les derniers tickets.

À dire à l'encadrant : Dire que l'admin a une vue globale de l'activité support.

[src/pages/admin/AdminTickets.jsx](#)

Rôle : Gestion des tickets.

Ce qu'on a fait : Liste tous les tickets, recherche par ticket/client/facture/description, filtre par statut et affiche une pagination visuelle.

À dire à l'encadrant : Dire que c'est la page de travail quotidienne de l'équipe support.

[src/pages/admin/AdminTicketDetail.jsx](#)

Rôle : Détail ticket admin.

Ce qu'on a fait : Marque le ticket comme lu, permet changement de statut, résolution, impression, note interne, réponse client, pièce jointe et historique.

À dire à l'encadrant : Dire que c'est la page la plus complète côté métier.

[src/pages/admin/AdminAnalytics.jsx](#)

Rôle : Analytique support.

Ce qu'on a fait : Calcule total, pending, seen, in progress, resolved, closed, activité et taux de résolution.

À dire à l'encadrant : Dire que cette page transforme les tickets en indicateurs de performance.

[src/pages/admin/AdminEmployees.jsx](#)

Rôle : Gestion employés.

Ce qu'on a fait : Appelle l'API réelle pour lister, créer et activer/désactiver les comptes employés/admin.

À dire à l'encadrant : Dire que cette partie dépend du backend car elle touche aux comptes internes.

14. Backend : configuration et serveur

[backend/package.json](#)

Rôle : Carte d'identité du backend.

Ce qu'on a fait : Déclare Express, Mongoose, JWT, bcrypt, Nodemailer, dotenv, helmet, cors, morgan et les scripts dev/start/admin:create.

À dire à l'encadrant : Dire que ce package est séparé du frontend pour gérer l'API.

[backend/package-lock.json](#)

Rôle : Versions exactes du backend.

Ce qu'on a fait : Généré automatiquement par npm.

À dire à l'encadrant : Dire qu'il garantit la reproductibilité des installations.

[backend/README.md](#)

Rôle : Documentation backend.

Ce qu'on a fait : Présente les endpoints principaux, la création d'un admin réel et l'intégration future avec le frontend.

À dire à l'encadrant : Dire qu'il explique comment passer du mode démo au mode API réelle.

[backend/.env.example](#)

Rôle : Exemple de variables d'environnement.

Ce qu'on a fait : Liste PORT, MONGO_URI, JWT_SECRET, CLIENT_URL, ADMIN_* et SMTP_*.

À dire à l'encadrant : Dire qu'il sert de modèle pour créer un .env sans exposer les secrets.

[backend/src/server.js](#)

Rôle : Point d'entrée de l'API.

Ce qu'on a fait : Configure Express, helmet, cors, JSON, morgan, route health, routes auth/employees/tickets, gestion d'erreur, connexion MongoDB et lancement du port.

À dire à l'encadrant : Dire que rien ne démarre sans connexion MongoDB valide.

[backend/src/config/database.js](#)

Rôle : Connexion MongoDB.

Ce qu'on a fait : Vérifie que MONGO_URI existe, active strictQuery et connecte Mongoose avec timeout.

À dire à l'encadrant : Dire que cette séparation rend la connexion réutilisable, notamment pour le script admin.

15. Backend : modèles MongoDB

[backend/src/models/User.js](#)

Rôle : Modèle utilisateur.

Ce qu'on a fait : Définit nom, email unique, passwordHash, société, rôle client/employee/admin, active, resetToken et dates automatiques.

À dire à l'encadrant : Dire que le mot de passe n'est jamais stocké en clair côté backend.

[backend/src/models/Ticket.js](#)

Rôle : Modèle ticket.

Ce qu'on a fait : Définit publicId, utilisateur, sujet, description, priorité, statut, infos client, historique, notes, messages, pièces jointes, lecture admin et timestamps.

À dire à l'encadrant : Dire que ce modèle représente toute la vie d'un ticket support.

16. Backend : sécurité et middlewares

[backend/src/middleware/auth.js](#)

Rôle : Middleware d'authentification.

Ce qu'on a fait : Lit le header Authorization Bearer, vérifie le JWT, charge l'utilisateur sans passwordHash et fournit req.user. requireRole limite certaines routes.

À dire à l'encadrant : Dire que le backend ne fait pas confiance au frontend : il revérifie le token et le rôle.

[backend/src/middleware/errorHandler.js](#)

Rôle : Gestion centralisée des erreurs.

Ce qu'on a fait : Renvoie un message propre et cache les détails pour les erreurs 500.

À dire à l'encadrant : Dire que cela évite de répéter try/catch dans la réponse finale.

17. Backend : contrôleurs

[backend/src/controllers/auth.controller.js](#)

Rôle : Logique d'authentification.

Ce qu'on a fait : Gère register, login, me, updateMe, forgotPassword et resetPassword avec bcrypt, JWT, token de reset et email.

À dire à l'encadrant : Dire que ce fichier contient les règles métier liées au compte.

[backend/src/controllers/ticket.controller.js](#)

Rôle : Logique métier des tickets.

Ce qu'on a fait : Crée un publicId, crée les tickets, liste selon rôle, change statut, ajoute notes/messages/pièces jointes, marque vu et envoie des emails.

À dire à l'encadrant : Dire que c'est le coeur du backend support.

[backend/src/controllers/employee.controller.js](#)

Rôle : Gestion des employés.

Ce qu'on a fait : Liste les admins/employés, crée un compte interne et le met à jour avec rôle, statut actif et mot de passe.

À dire à l'encadrant : Dire que seuls les admins peuvent utiliser ces actions.

18. Backend : routes

[backend/src/routes/auth.routes.js](#)

Rôle : Routes auth.

Ce qu'on a fait : Associe /register, /login, /forgot-password, /reset-password, /me et PATCH /me aux contrôleurs.

À dire à l'encadrant : Dire que les routes /me sont protégées par requireAuth.

[backend/src/routes/ticket.routes.js](#)

Rôle : Routes tickets.

Ce qu'on a fait : Protège toutes les routes par `requireAuth`, puis limite `seen/status/notes` aux employés/admins.

À dire à l'encadrant : Dire que le client peut créer/lire ses tickets et ajouter des messages/pièces jointes, mais pas changer le statut.

[backend/src/routes/employee.routes.js](#)

Rôle : Routes employés.

Ce qu'on a fait : Toutes les routes exigent `requireAuth` et `requireRole('admin')`.

À dire à l'encadrant : Dire que la gestion du personnel est réservée à l'administrateur.

19. Backend : services et scripts

[backend/src/services/email.service.js](#)

Rôle : Service email.

Ce qu'on a fait : Vérifie la configuration SMTP, crée le transport Nodemailer, envoie les emails ou log l'email si SMTP n'est pas configuré. `appUrl` construit les liens frontend.

À dire à l'encadrant : Dire que le projet ne plante pas si SMTP manque : il saute l'envoi proprement.

[backend/src/scripts/create-admin.js](#)

Rôle : Script de création admin.

Ce qu'on a fait : Lit les variables `ADMIN_*`, connecte MongoDB, hash le mot de passe et crée/met à jour le compte admin.

À dire à l'encadrant : Dire que c'est la méthode propre pour créer le vrai compte administrateur.

20. Assets et fichiers publics

[src/assets/pbxcom-logo-dark.png](#)

Rôle : Logo utilisé en thème sombre.

Ce qu'on a fait : Affiché par BrandLogo quand la classe dark est active.

À dire à l'encadrant : Dire que le logo s'adapte au thème.

[src/assets/pbxcom-logo-light.png](#)

Rôle : Logo utilisé en thème clair.

Ce qu'on a fait : Affiché par BrandLogo en mode clair.

À dire à l'encadrant : Dire que cela garde une bonne lisibilité.

[src/assets/hero.png](#)

Rôle : Image de support visuel.

Ce qu'on a fait : Présente dans les assets, prête pour une page ou un écran visuel.

À dire à l'encadrant : Dire qu'elle fait partie des ressources graphiques.

[src/assets/react.svg](#) et [src/assets/vite.svg](#)

Rôle : Assets du template initial.

Ce qu'on a fait : Encore présents mais pas essentiels au fonctionnement PBxcom.

À dire à l'encadrant : Dire qu'ils peuvent être supprimés plus tard si on nettoie le projet.

[public/favicon.png](#) et [public/favicon-64.png](#)

Rôle : Icônes de l'onglet navigateur.

Ce qu'on a fait : Utilisées par index.html ou le navigateur pour l'identité visuelle.

À dire à l'encadrant : Dire que ce sont les petites icônes du site.

[public/icons.svg](#)

Rôle : Fichier SVG public.

Ce qu'on a fait : Ressource publique disponible depuis le navigateur.

À dire à l'encadrant : Dire qu'il peut servir d'asset graphique global.

[.DS_Store](#)

Rôle : Fichiers système macOS.

Ce qu'on a fait : Créés automatiquement par Finder, sans rôle dans l'application.

À dire à l'encadrant : Dire qu'ils doivent idéalement être ignorés/supprimés du dépôt.

21. Parcours à expliquer à l'encadrant

Parcours client

- Le client se connecte ou crée un compte.

- Il ouvre Nouveau ticket.
- Il remplit ses informations : nom, prénom, société, numéro de marché, facture, téléphone, email, ville et description du problème.
- Il ajoute des pièces jointes si nécessaire.
- Le système crée un ticket numéroté et l'ajoute à son historique.
- Le client peut ensuite ouvrir le détail, suivre le statut et répondre.

Parcours admin/employé

- L'admin se connecte avec le compte PBxcom.
- Il voit les statistiques et la liste des tickets.
- Il ouvre un ticket : le système le marque comme vu.
- Il peut répondre au client, ajouter une note interne, joindre un fichier, changer le statut ou imprimer la fiche.
- Les changements apparaissent dans l'historique du ticket.

22. Questions probables et réponses

Q : Pourquoi React ?

R : Parce que l'interface est composée de pages et composants réutilisables : formulaires, tableaux, badges, layouts. React rend cette organisation claire.

Q : Pourquoi Zustand ?

R : Pour partager facilement l'utilisateur connecté, les tickets, les notifications, la langue et le thème entre plusieurs pages sans passer les données manuellement partout.

Q : Où sont stockées les données ?

R : En mode démonstration, elles sont dans le localStorage du navigateur via Zustand persist. En mode production, le backend est prévu pour stocker les données dans MongoDB.

Q : Pourquoi il y a un backend si le frontend marche seul ?

R : Le frontend seul facilite la démonstration. Le backend prépare la vraie version : MongoDB, JWT, rôles, emails et comptes employés.

Q : Comment la sécurité est gérée ?

R : Côté frontend, ProtectedRoute bloque les pages selon le rôle. Côté backend, requireAuth vérifie le token JWT et requireRole vérifie les autorisations.

Q : Pourquoi les mots de passe sont sécurisés ?

R : Dans le backend, ils sont hashés avec bcrypt. On ne stocke pas le mot de passe original.

Q : Comment les emails marchent ?

R : Le backend utilise Nodemailer avec SMTP. Si SMTP n'est pas configuré, le service n'envoie pas l'email mais ne fait pas planter l'application.

Q : Comment le ticket est tracé ?

R : Chaque ticket contient history, messages, notes et timestamps. Les changements de statut et messages ajoutent des événements.

Q : Quelle différence entre note et message ?

R : Le message est une conversation visible client/support. La note interne est réservée à l'équipe support.

Q : Pourquoi utiliser publicId ?

R : Pour afficher un numéro simple comme 00001 au lieu de montrer l'identifiant MongoDB technique.

Q : Qu'est-ce qui reste à améliorer ?

R : Brancher définitivement le frontend sur l'API, ajouter validation backend avec Zod, pagination réelle, tests automatisés et nettoyage des fichiers template/.DS_Store.

23. Points forts du projet

- Séparation claire entre interface, état, services, backend et base de données.
- Deux espaces distincts : client et admin/employé.
- Gestion complète du cycle d'un ticket : création, lecture, statut, conversation, notes, pièces jointes, impression.
- Mode démonstration autonome sans base de données.
- Backend prêt pour une vraie mise en production avec MongoDB et emails.
- Interface responsive avec thème sombre/clair et support multilingue.

24. Limites à connaître honnêtement

- Le frontend utilise encore un fallback local ; pour une production complète, il faut forcer l'utilisation de l'API.
- Les notifications frontend sont simulées, même si le backend contient l'envoi email réel.
- La pagination dans AdminTickets est seulement visuelle pour l'instant.
- Certains fichiers du template initial comme react.svg/vite.svg ne sont pas nécessaires.
- Il faut ajouter plus de validation backend pour tous les champs de formulaire.

25. Conclusion simple à dire

Ce projet est une plateforme de gestion de tickets support pour PBxcom. Le client crée et suit ses demandes. L'équipe PBxcom traite ces demandes, change leur statut, répond, ajoute des notes internes et peut imprimer une fiche. Le frontend est prêt pour la démonstration grâce au stockage local, et le backend prépare la version réelle avec MongoDB, JWT, rôles et emails.

Phrase courte pour la soutenance :

« J'ai développé une application web de support qui centralise les demandes clients sous forme de tickets, avec un espace client, un espace administrateur, un suivi de statut, des conversations, des pièces jointes, des notifications et une architecture prête pour une base MongoDB. »